Two quick exams, then: how to beat Friday

TC1037 · Algorithmic Thinking · Session 9 · midterm review

Dr. Leopoldo Rodríguez Salazar

Tuesday 20 October 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

Two minutes, two requests

INE — credencial y voto informado

FETEC's one-minute survey: who already has an INE and who needs one for the 2027 elections.

`forms.cloud.microsoft/r/ZrqcPUndjT`

Do it now, on your phone.

Levelling-module survey (IFE)

12 minutes, personal link in your Tec email — search *Instituto para el Futuro de la Educación* (check spam).

Closes Friday 23 October.

Friday 23 October — the midterm, on paper

Part I — read and predict · 30 pts

Six short programs: arithmetic · boolean · conditional · functions · `while` · `for`.
Write *exactly* what Python prints — every line, in order.
5 points each, all or nothing.

Part II — write the program · 70 pts

a function with a `while`	23
a function with a `for`	23
`menu()`	3
`main()`: loop, validate, call, print	20
last line calls `main()`	1

80 minutes · no computer, phone, notes or calculator · pen · versions A and B

Three practice papers with solutions — on the site now (Exams page).

Part I — read the operators first: who goes first?

1	( ) parentheses
2	** power
3	* / // % — left to right
4	+ - — left to right
5	== != < > <= >= → True/False
6	not
7	and
8	or
last	= stores the result in the variable

What each arithmetic operator gives

6 / 3 → 2.0	/ *always* gives a float
15 // 4 → 3	how many whole times 4 fits
15 % 4 → 3	what is left over
2 ** 3 → 8	power
1234 % 10 → 4	the last digit
1234 // 10 → 123	drop the last digit

The method: find the operator that goes first, compute it, *rewrite the line*, repeat. One step per line, on paper.

+ - * / // % ** arithmetic == != < > <= >= comparison not and or boolean = stores a value

Part I — loops: a table, one row per turn

```
n = 50
count = 0
while n > 5:
    n = n // 2
    count = count + 1
print(n, count)
```

turn	n > 5?	n	count	
start	—	50	0	
1	yes	25	1	50 // 2
2	yes	12	2	25 // 2 drops the .5
3	yes	6	3	
4	yes	3	4	
5	**no**			3 > 5 is False → stop

How to fill it:

1. Row *start*: the values before the loop.
2. Check the test *with the current values*.
3. *yes* → run the body top to bottom, write the new values.
4. The first *no* ends the loop. `print` uses the last row.

Part I — loops: a table, one row per turn

```
n = 50
count = 0
while n > 5:
    n = n // 2
    count = count + 1
print(n, count)
```

How to fill it:

1. Row *start*: the values before the loop.
2. Check the test *with the current values*.
3. *yes* → run the body top to bottom, write the new values.
4. The first *no* ends the loop. `print` uses the last row.

turn	`n > 5`**?**	`n`	`count`	
start	—	50	0	
1	yes	25	1	`50 // 2`
2	yes	12	2	`25 // 2` drops the .5
3	yes	6	3	
4	yes	3	4	
5	**no**			`3 > 5` is False → stop

Output: `3 4`

The test is checked *before* every turn — including the one that stops it. Practice 1, question 5.

Trap 1 — precedence: one step at a time

```
print(10 - 6 / 3 * 2 + 15 // 4)
```

```
10 - 6 / 3 * 2 + 15 // 4
```

Trap 1 — precedence: one step at a time

```
print(10 - 6 / 3 * 2 + 15 // 4)
```

```
  10 - 6 / 3 * 2 + 15 // 4
= 10 - 2.0 * 2 + 15 // 4
```

`/` first (leftmost of `* / // %`) — and it gives a **float**

Trap 1 — precedence: one step at a time

```
print(10 - 6 / 3 * 2 + 15 // 4)
```

```
    10 - 6 / 3 * 2 + 15 // 4
=   10 - 2.0 * 2 + 15 // 4      / first (leftmost of * / // %) — and
                                it gives a float
=   10 - 4.0 + 15 // 4          then *: 2.0 * 2 = 4.0
```

Trap 1 — precedence: one step at a time

```
print(10 - 6 / 3 * 2 + 15 // 4)
```

```
    10 - 6 / 3 * 2 + 15 // 4
=   10 - 2.0 * 2 + 15 // 4      / first (leftmost of * / // %) — and
                                it gives a float
=   10 - 4.0 + 15 // 4          then *: 2.0 * 2 = 4.0
=   10 - 4.0 + 3                then //: 4 fits in 15 three whole times
```

Trap 1 — precedence: one step at a time

```
print(10 - 6 / 3 * 2 + 15 // 4)
```

```
    10 - 6 / 3 * 2 + 15 // 4
=   10 - 2.0 * 2 + 15 // 4      / first (leftmost of * / // %) — and
                                it gives a float
=   10 - 4.0 + 15 // 4          then *: 2.0 * 2 = 4.0
=   10 - 4.0 + 3                then //: 4 fits in 15 three whole times
=   6.0 + 3                     now + −, left to right
```

Trap 1 — precedence: one step at a time

```
print(10 - 6 / 3 * 2 + 15 // 4)
```

```
    10 - 6 / 3 * 2 + 15 // 4
=   10 - 2.0 * 2 + 15 // 4      / first (leftmost of * / // %) — and
                                it gives a float
=   10 - 4.0 + 15 // 4          then *: 2.0 * 2 = 4.0
=   10 - 4.0 + 3                then //: 4 fits in 15 three whole times
=   6.0 + 3                     now + −, left to right
=   9.0                         a float plus an int is a float
```

Trap 1 — precedence: one step at a time

```
print(10 - 6 / 3 * 2 + 15 // 4)
```

	`10 - 6 / 3 * 2 + 15 // 4`	
=	`10 - 2.0 * 2 + 15 // 4`	`/` first (leftmost of `* / // %`) — and it gives a **float**
=	`10 - 4.0 + 15 // 4`	then `*`: `2.0 * 2` = `4.0`
=	`10 - 4.0 + 3`	then `//`: 4 fits in 15 three whole times
=	`6.0 + 3`	now `+ -`, left to right
=	`9.0`	a float plus an int is a float

Output: `9.0` — not 9, not 9.75. One `/` anywhere makes the result a float.

`+ - * / // % **` arithmetic `== != < > <= >=` comparison `not and or` boolean `=` stores a value

Trap 2 — boolean order: comparisons, then `not`, `and`, `or`

```
print(4 > 3 and not 2 == 2.0 or 7 != 7)
```

```
4 > 3 and not 2 == 2.0 or 7 != 7
```

Trap 2 — boolean order: comparisons, then `not`, `and`, `or`

```
print(4 > 3 and not 2 == 2.0 or 7 != 7)
```

```
    4 > 3 and not 2 == 2.0 or 7 != 7
=   True and not True or False          all comparisons first; 2 == 2.0 is
                                        True (same value)
```

Trap 2 — boolean order: comparisons, then `not`, `and`, `or`

```
print(4 > 3 and not 2 == 2.0 or 7 != 7)
```

	`4 > 3 and not 2 == 2.0 or 7 != 7`	
=	`True and not True or False`	all comparisons first; `2 == 2.0` is True (same value)
=	`True and False or False`	then `not`: flips only the value right after it

Trap 2 — boolean order: comparisons, then `not`, `and`, `or`

```
print(4 > 3 and not 2 == 2.0 or 7 != 7)
```

	`4 > 3 and not 2 == 2.0 or 7 != 7`	
=	`True and not True or False`	all comparisons first; `2 == 2.0` is True (same value)
=	`True and False or False`	then `not`: flips only the value right after it
=	`False or False`	then `and`: True and False is False

Trap 2 — boolean order: comparisons, then `not`, `and`, `or`

```
print(4 > 3 and not 2 == 2.0 or 7 != 7)
```

	`4 > 3 and not 2 == 2.0 or 7 != 7`	
=	`True and not True or False`	all comparisons first; `2 == 2.0` is True (same value)
=	`True and False or False`	then `not`: flips only the value right after it
=	`False or False`	then `and`: True and False is False
=	`False`	`or` last: False or False is False

Trap 2 — boolean order: comparisons, then `not`, `and`, `or`

```
print(4 > 3 and not 2 == 2.0 or 7 != 7)
```

	`4 > 3 and not 2 == 2.0 or 7 != 7`	
=	`True and not True or False`	all comparisons first; `2 == 2.0` is True (same value)
=	`True and False or False`	then `not`: flips only the value right after it
=	`False or False`	then `and`: True `and` False is False
=	`False`	`or` last: False `or` False is False

`and`: True only if *both* are True `or`: True if *at least one* is True `not`: the opposite

`+ - * / // % **` arithmetic `== != < > <= >=` comparison `not and or` boolean `=` stores a value

Trap 3 — functions: every function has its own variables

```
def h(x, y):
    x = x + y
    y = x - y
    return x * y

def main():
    x = 2
    y = 3
    z = h(y, x)
    print(x, y, z)

main()
```

	main			h	
step	x	y	z	x	y
x = 2, y = 3	2	3			

Trap 3 — functions: every function has its own variables

```
def h(x, y):
    x = x + y
    y = x - y
    return x * y

def main():
    x = 2
    y = 3
    z = h(y, x)
    print(x, y, z)

main()
```

	main			h	
step	x	y	z	x	y
x = 2, y = 3	2	3			
call h(y, x)	2	3		3	2

Trap 3 — functions: every function has its own variables

```
def h(x, y):
    x = x + y
    y = x - y
    return x * y

def main():
    x = 2
    y = 3
    z = h(y, x)
    print(x, y, z)

main()
```

	main			h	
step	x	y	z	x	y
x = 2, y = 3	2	3			
call h(y, x)	2	3		3	2
x = x + y				5	2
y = x - y				5	3

Trap 3 — functions: every function has its own variables

```
def h(x, y):
    x = x + y
    y = x - y
    return x * y

def main():
    x = 2
    y = 3
    z = h(y, x)
    print(x, y, z)

main()
```

	main			h	
step	x	y	z	x	y
x = 2, y = 3	2	3			
call h(y, x)	2	3		3	2
x = x + y				5	2
y = x - y				5	3
return x * y	2	3	15	gone	

Trap 3 — functions: every function has its own variables

```
def h(x, y):
    x = x + y
    y = x - y
    return x * y

def main():
    x = 2
    y = 3
    z = h(y, x)
    print(x, y, z)

main()
```

	main			h	
step	x	y	z	x	y
x = 2, y = 3	2	3			
call h(y, x)	2	3		3	2
x = x + y				5	2
y = x - y				5	3
return x * y	2	3	15	gone	

Output: 2 3 15

Arguments go **by position, not by name**: main's y (3) lands in h's x. Changing h's x and y never touches main's — they are different boxes.

Trap 4 — `range`: write the values before you trace

`range(start, stop, step)`: **start** is included, **stop** never is, **step** is the jump.

`range(1, 12, 4)` → 1, 5, 9
13 would pass 12: stop

`range(5, 0, -2)` → 5, 3, 1
counting down, stops before 0

`range(1, n + 1)` → 1, 2, . . . , n
the `+ 1` is what includes n

`range(4)` → 0, 1, 2, 3
four values, starting at 0

```
total = 0
for i in range(5, 0, -2):
    total = total + i
    print(total)
```

i	total	prints
5	5	5
3	8	8
1	9	9

Trap 4 — `range`: write the values before you trace

`range(start, stop, step)`: **start** is included, **stop** never is, **step** is the jump.

`range(1, 12, 4)` → 1, 5, 9
13 would pass 12: stop

`range(5, 0, -2)` → 5, 3, 1
counting down, stops before 0

`range(1, n + 1)` → 1, 2, . . . , n
the `+ 1` is what includes n

`range(4)` → 0, 1, 2, 3
four values, starting at 0

```
total = 0
for i in range(5, 0, -2):
    total = total + i
    print(total)
```

i	total	prints
5	5	5
3	8	8
1	9	9

Output: three lines, `5` / `8` / `9`. The `print` is *inside* the loop: one line per turn.

`for` in the midterm — three places, one template

Where you will meet it

1. Part I, Q6 — trace a `for` (5 pts)
2. Part I, Q4 — a function with a `for` *inside* (5 pts)
3. Part II, function 8 — **must** be written with `for` (23 pts)

Checklist before you hand in

- start value: `0` for a sum or a count, `1` for a product
- the end: `range(1, n + 1)` if `n` must be included
- `return` **after** the loop, not inside it

```
# every Part II for-function
def name(n):
    result = 0          # product: 1
    for i in range(1, n + 1):
        if i % 2 == 0:  # filter?
            result = result + i
    return result       # after
```

`for` in the midterm — three places, one template

Where you will meet it

1. Part I, Q6 — trace a `for` (5 pts)
2. Part I, Q4 — a function with a `for` *inside* (5 pts)
3. Part II, function 8 — **must** be written with `for` (23 pts)

Checklist before you hand in

- start value: `0` for a sum or a count, `1` for a product
- the end: `range(1, n + 1)` if `n` must be included
- `return` **after** the loop, not inside it

```
# every Part II for-function
def name(n):
    result = 0          # product: 1
    for i in range(1, n + 1):
        if i % 2 == 0:  # filter?
            result = result + i
    return result       # after
```

The mistake that costs the most:

```
    for i in range(1, n + 1):
        result = result + i
        return result  # 1 turn!
```

Indented under the `for`, the `return` leaves the function in the *first* turn.

Part II — write it in this order

1. **The two functions first** — 46 of the 70 points. Header, accumulator, loop, `return`. No `input`, no `print`.
2. **Test them in your head** with the examples on the paper — *especially the smallest one* (`0`, `1`).
3. `menu()` — three `print`s. Three points; do not spend time on it.
4. `main()` — the skeleton you have written in every lab since week 4 →

```
def main():
    keep_going = True
    while keep_going:
        menu()
        option = int(input("Option: "))
        if option == 1:
            ...  # 4 steps
        elif option == 2:
            ...  # same
        elif option == 3:
            print("Goodbye.")
            keep_going = False
        else:
            print("Invalid option.")
            keep_going = False  # ends
main()
```

Practice 1, live: `sum_digits(n)` with `while`

```
# sum of the digits of n
def sum_digits(n):
    total = 0
    while n > 0:
        total = total + n % 10
        n = n // 10
        # n loses a digit
    return total
```

`n % 10` is the last digit; `n // 10` drops it.

`n`	`n > 0`?	`n % 10`	`total`
1234	yes	4	4
123	yes	3	7
12	yes	2	9
1	yes	1	10
0	**no**		$\rightarrow$ **10**

Ask yourself every time: what does my loop do with the *smallest* legal input? Here 0: zero turns, returns 0 — right. *On Friday it may not be.*

Practice 1, live: `count_multiples(n, k)` with `for` — and `main`'s option 2

```
# multiples of k in 1..n
def count_multiples(n, k):
    count = 0
    for i in range(1, n + 1):
        if i % k == 0:
            count = count + 1
    return count
```

```
elif option == 2:
    n = int(input("n (1 or more): "))
    k = int(input("k (1 or more): "))
    if n < 1 or k < 1:
        print("n and k must be 1 or more.")
    else:
        print("Multiples:",
              count_multiples(n, k))
```

Read · validate · call · print *with a label*. Four lines of thought, every option.

`range(1, n + 1)`: the `+ 1` is written every time, forever. `count_multiples(12, 3)` → 4, not 3.

How Part II is graded

Excelente 100%	no syntax errors; works, *including the smallest examples*
Muy bien 85%	right approach; one small slip (the `+ 1`, the `0` case)
Regular 70%	at most two syntax errors and/or small logic errors; clearly the right idea
Deficiente 50%	loosely related, or more than two syntax errors
Muy deficiente 20%	code unrelated to the problem
Sin evidencia 0	nothing written

Per part: $23 \cdot 23 \cdot 3 \cdot 20 \cdot 1$. Then -2 for each missing: meaningful names · a prompt for every input · labelled output.

On paper, syntax still counts: colons, indentation, parentheses. Write it as if Thonny were reading it.

Before Friday

1. **Practice midterms 1, 2 and 3** — site → Exams. Solve on paper, 80 minutes, *then* open the solutions.
2. **Friday 16:30 Lab 5** with Luis — the academia's *for y while* lab 2 (primes, perfect numbers). Then **17:35 the midterm**, Room 4206. Bring a pen.
3. **Homework 5** — Sunday 25 October, 23:59.
4. QE3: today's sitting is your QE3 grade; the 13 Oct attempt can only replace a lower quick exam.
5. **Situación problema:** join your team in Canvas (People → Groups) if you have not. Dataset: Friday 6 November.

Next week is Tec Week — no classes. We come back Tuesday 3 November with lists.

The smallest input first.

Then the `+ 1`. Then `main`.